

CapFinLoan

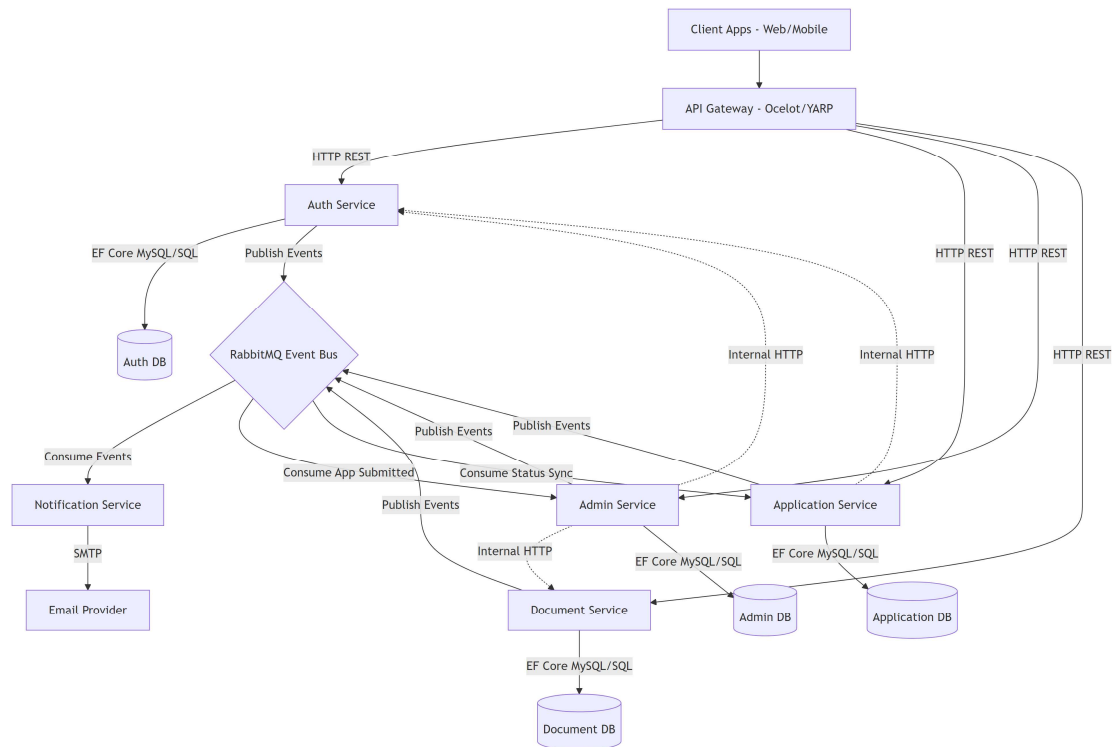
Financial Loan Management System

Low-Level Design (LLD)

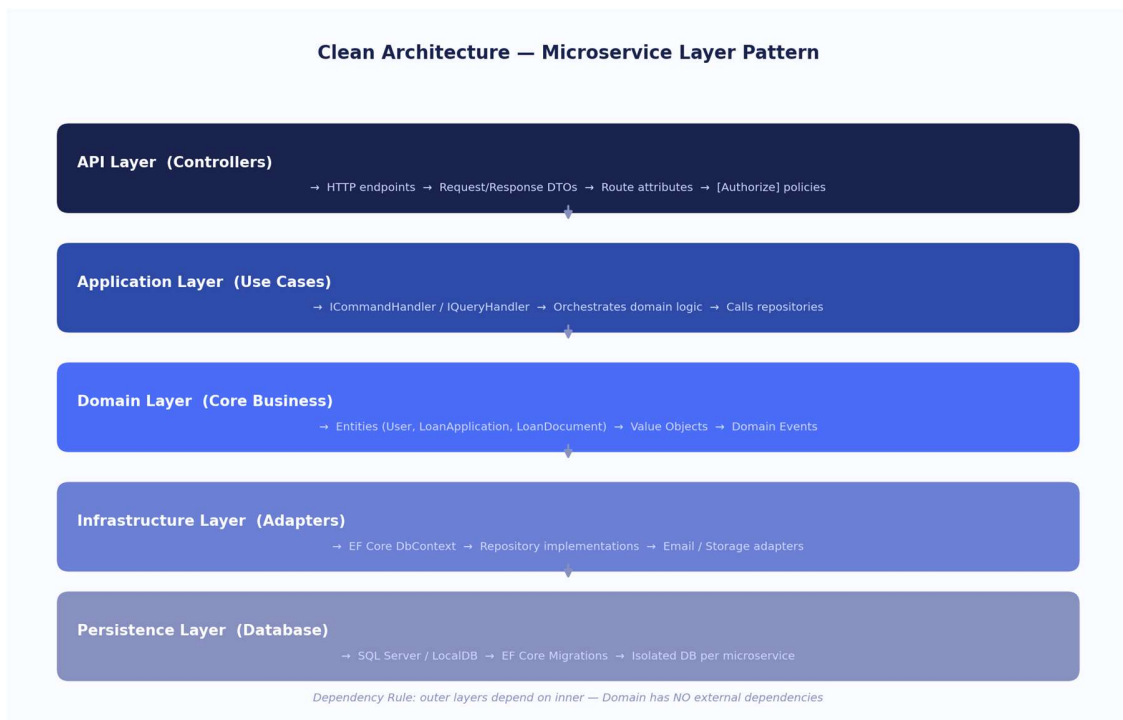
Backend: ASP.NET Core 6 Microservices | Frontend: Angular 21

Version 1.0 | May 2026 | Complete & Authoritative

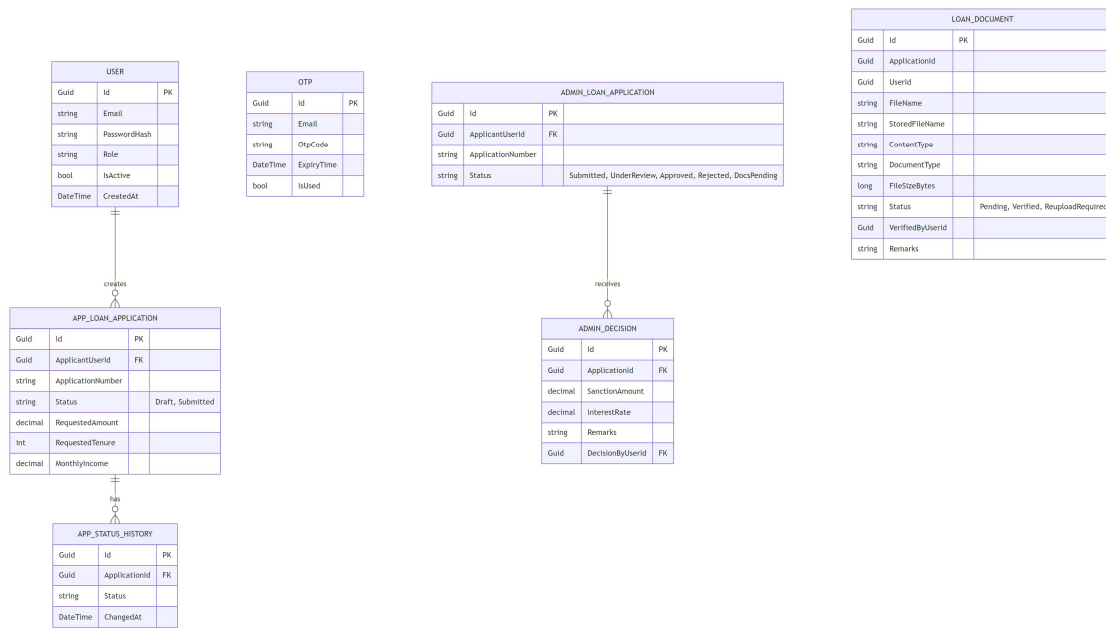
1. System Overview



System Architecture



Clean Architecture Layers



Database Entity Relationship Diagram

CapFinLoan is a full-stack loan management platform composed of five ASP.NET Core 10 backend microservices, one FastAPI chatbot service, and an Angular 21 SPA frontend, all fronted by an Ocelot API Gateway.

Each domain service owns its own SQL Server database schema. Inter-service asynchronous communication uses RabbitMQ with MassTransit. Authentication is JWT bearer based, and each service validates JWT independently using shared issuer, audience, and signing key configuration.

1.1 Architecture Summary

Component	Technology	Port	Responsibility
Angular SPA	Angular 21 / TypeScript	4200	Applicant and Admin UI, guarded routes, wallet UI, chatbot UI integration
API Gateway	Ocelot + SwaggerForOcelot	5020	Single API ingress, route forwarding, CORS, aggregated swagger
Auth Service	ASP.NET Core 10 + Identity	5021	Signup/login, OTP, JWT generation, admin user management
Application Service	ASP.NET Core 10 + EF Core	5022	Applicant profile, loan drafts/submission, status history, wallet and payment orchestration
Document Service	ASP.NET Core 10 + EF Core	5023	Document upload, replace, link,

			download, verification lifecycle
Admin Service	ASP.NET Core 10 + EF Core	5024	Queue/dashboard, decision updates, user and document admin proxy APIs
Notification Service	ASP.NET Core 10 + MassTransit	5025	Email notifications and failure signaling consumers
Chatbot Service	FastAPI + Groq SDK	5026	JWT-aware chat orchestration, applicant guided flow, admin AI summaries
RabbitMQ	rabbitmq:3-management	5673 / 15673	Event bus and consumer queues
SQL Server	mssql/server:2022-latest	1433	Isolated service databases

1.2 Cross-Cutting Concerns

- Authentication: JWT bearer across services, roles APPLICANT and ADMIN.
- Error Handling: global exception middleware in each .NET API service.
- Eventing: MassTransit with service-specific endpoint naming formatters.
- Configuration: appsettings + environment variables + DotNetEnv for local env loading.
- Persistence pattern: clean layering with Domain, Application, Infrastructure, Persistence projects.

2. Shared Contracts Library (CapFinLoan.Messaging.Contracts)

The shared contracts project provides strongly typed event contracts consumed by multiple services.

2.1 Common Response DTOs

Service APIs primarily return typed JSON DTOs and domain-specific payloads. Error responses are standardized through middleware and service exceptions.

2.2 Domain Events (RabbitMQ Messages)

Event	Key Fields	Typical Publisher to Consumers
UserRegisteredEvent	UserId, Email, FullName, Role, RegisteredAtUtc	Auth -> Application, Notification
ApplicationSubmittedEvent	ApplicationId, ApplicantUserId, ApplicationNumber, RequestedAmount	Application -> Admin, Notification
ApplicationStatusChangedEvent	ApplicationId, PreviousStatus, NewStatus, ChangedByUserId	Admin or Application sync -> Notification, Application/Admin sync

		consumers
ApplicationStatusRolledBackEvent	ApplicationId, PreviousStatus, RolledBackFromStatus, Remarks	Admin rollback flow -> Notification/consumers
StatusSyncFailedEvent	ApplicationId, AttemptedStatus, FailureReason	Application -> Admin compensation consumer
DocumentVerifiedEvent	DocumentId, ApplicationId, IsVerified, Remarks, VerifiedByUserId	Document -> Admin, Application, Notification
LoanApprovedEvent	ApplicationId, SanctionAmount, ApprovedByUserId	Admin/Application process -> Notification
LoanRejectedEvent	ApplicationId, ApplicantName, Remarks, RejectedByUserId	Admin/Application process -> Notification
OtpSendEvent	Email, OtpCode, Purpose, ExpiresAtUtc	Auth -> Notification
NotificationFailedEvent	ApplicationId, NotificationType, FailureReason	Notification -> Admin consumer for ops visibility

2.3 Exception Hierarchy

Each service has an ApplicationServiceException-style pattern with specific derived exceptions. Notable wallet-specific exception:

- InsufficientWalletBalanceException with HTTP 402 and error code INSUFFICIENT_WALLET_BALANCE.

2.4 Constants

Representative constants:

- Application statuses: Draft, Submitted, Docs Pending, Docs Verified, Under Review, Approved, Rejected, Closed.
- Wallet owner types: Applicant, Admin.
- Wallet directions: Credit, Debit.
- Wallet entry types: TopUp, ApplicationFee, LoanDisbursal, LoanRepayment, Compensation, Withdrawal.
- Document status enum: Pending, UnderReview, Verified, ReuploadRequired.

3. API Gateway (Ocelot)

Port: 5020 | Technology: Ocelot + SwaggerForOcelot

The gateway performs route forwarding and swagger aggregation. JWT validation remains in downstream services.

3.1 Route Table (ocelot.json)

Upstream Path	Downstream Path	Downstream Service	Methods
/gateway/auth/{everything}	/api/auth/{everything}	Auth (5021 local / 8080 docker)	GET, POST, PUT, DELETE
/gateway/applications/{everything}	/api/applications/{everything}	Application (5022 / 8080)	GET, POST, PUT, DELETE
/gateway/documents/{everything}	/api/documents/{everything}	Document (5023 / 8080)	GET, POST, PUT, DELETE
/gateway/admin/{everything}	/api/admin/{everything}	Admin (5024 / 8080)	GET, POST, PUT, DELETE
/gateway/chat	/chat	Chatbot (5026 / 8000)	POST
/gateway/chat/{everything}	/everything	Chatbot (5026 / 8000)	GET, POST, DELETE
/gateway/api/{everything}	/api/{everything}	Gateway self route	GET, POST, PUT, DELETE

3.2 CORS Policy

Gateway currently allows any origin, any method, and any header through default policy.

3.3 Swagger Aggregation

Gateway aggregates swagger docs for:

- Auth Service
- Application Service
- Document Service
- Admin Service

4. Auth Service

Port: 5021 | Route prefix: api/auth and api/internal/users

4.1 Data Models

Primary entity:

- ApplicationUser (IdentityUser<Guid>) with Name, IsActive, CreatedAtUtc, UpdatedAtUtc.

Supporting entity:

- EmailVerificationOtp with Email, OtpCode, ExpiresAtUtc, IsUsed.

4.2 API Endpoints

Verb	Path	Auth	Description
POST	/api/auth/signup	None	Applicant signup initiation
POST	/api/auth/signup-admin	None	Admin signup initiation
POST	/api/auth/send-otp?email=	None	Send signup OTP
POST	/api/auth/forgot-password/send-otp?email=	None	Send forgot password OTP
POST	/api/auth/verify-otp-signup	None	Verify OTP and complete applicant signup
POST	/api/auth/verify-otp-signup-admin	None	Verify OTP and complete admin signup
POST	/api/auth/login	None	Credentials login and JWT issuance
POST	/api/auth/google-login	None	Google ID token login
POST	/api/auth/forgot-password/reset	None	OTP-based password reset
GET	/api/internal/users	Admin	List users for admin panel
GET	/api/internal/users/{id}/notification-info	Internal key (anonymous endpoint with key check)	Notification profile lookup
PUT	/api/internal/users/{id}/status	Admin	Activate/deactivate user

4.3 Service Layer

- AuthService: signup, OTP verification, login, password reset, user status operations.
- JwtTokenGenerator: token creation with issuer, audience, key.
- OtpRepository and Otp flow: OTP persistence and expiry validation.
- RabbitMqEventPublisher: publishes user and OTP-related events.

5. Application Service

Port: 5022 | Route prefix: api/applications

5.1 Data Models

Core entities:

- LoanApplication
- ApplicationStatusHistory
- ApplicantProfile
- WalletAccount
- WalletLedgerEntry
- PaymentOrder

5.2 API Endpoints

Verb	Path	Auth	Description
GET	/api/applications/profile	Applicant	Get applicant profile
PUT	/api/applications/profile	Applicant	Save applicant profile
GET	/api/applications/my	Authenticated	List current user applications
GET	/api/applications/{id}	Authenticated	Get application by id
POST	/api/applications	Applicant	Create draft application
PUT	/api/applications/{id}	Applicant	Update draft application
POST	/api/applications/{id}/submit	Applicant	Submit draft application
GET	/api/applications/{id}/status	Authenticated	Get current status and timeline
DELETE	/api/applications/{id}	Applicant	Delete draft
GET	/api/applications/wallet/summary	Applicant	Applicant wallet summary
GET	/api/applications/wallet/ledger?take=	Applicant	Applicant wallet ledger
POST	/api/applications/wallet/topup/create-order	Applicant	Create applicant topup order
POST	/api/applications/wallet/topup/verify	Applicant	Verify applicant topup payment
POST	/api/applications/wallet/withdraw	Applicant	Applicant withdraw operation
GET	/api/applications/wallet/config	Applicant	Wallet configuration including application fee

GET	/api/applications/wallet/admin/summary	Admin	Admin wallet summary
POST	/api/applications/wallet/admin/topup/create-order	Admin	Create admin topup order
POST	/api/applications/wallet/admin/topup/verify	Admin	Verify admin topup payment

5.3 Core Logic: Loan, Wallet, and EMI

- Draft -> submit lifecycle with validation and ownership checks.
- Profile completeness and status timeline support.
- Wallet fee deduction on submission.
- Admin disbursal balance checks.
- Explicit insufficient wallet handling via dedicated custom exception.

5.4 Service Layer

- LoanApplicationService for profile/application lifecycle operations.
- WalletService for account summary, ledger, topup verification, and debit/credit logic.
- Repositories for profile, loan, and wallet persistence.
- MassTransit consumers:
- ApplicationStatusChangedConsumer
- UserRegisteredConsumer
- DocumentVerifiedConsumer

6. Document Service

Port: 5023 | Route prefixes: api/documents and api/internal/documents

6.1 Data Models

Primary model:

- LoanDocument: file metadata, document type, status enum, verification metadata, timestamps.

6.2 Document Lifecycle

Status	Meaning
Pending	Uploaded and waiting for review
UnderReview	Being reviewed by admin
Verified	Accepted by admin
ReuploadRequired	Rejected and requires fresh upload

6.3 Validation and Guards

- Ownership checks for applicant-only operations.
- Role checks for admin-only verification endpoints.

- Replace/link/download constraints and status updates are service enforced.

6.4 API Endpoints

Verb	Path	Auth	Description
POST	/api/documents/upload	Applicant	Upload document by application and type
GET	/api/documents/application/{applicationId}	Authenticated	List docs for application
GET	/api/documents/my	Authenticated	List current user docs
GET	/api/documents/{id}	Authenticated	Get doc metadata
PUT	/api/documents/{id}	Applicant	Replace uploaded file
POST	/api/documents/{id}/link	Applicant	Link existing document to another application
GET	/api/documents/{id}/download	Applicant	Download/view own document
PUT	/api/internal/documents/{id}/verify	Admin	Verify or reject document
GET	/api/internal/documents/application/{applicationId}	Admin	List docs for application
GET	/api/internal/documents/all?status=	Admin	List all docs with optional filter
GET	/api/internal/documents/{id}/download	Admin	Download any document

6.5 Service Layer

- DocumentService combines document repository and application-state integration.
- Local file storage implementation under wwwroot/uploads.
- TokenForwardingHandler used in internal service-to-service calls.

7. Admin Service

Port: 5024 | Route prefixes: api/admin/applications, api/admin/documents, api/admin/users

7.1 Data Models

Primary entities:

- LoanApplication (admin projection)
- ApplicationStatusHistory

- Decision (sanction amount, interest rate, remarks, decision status)

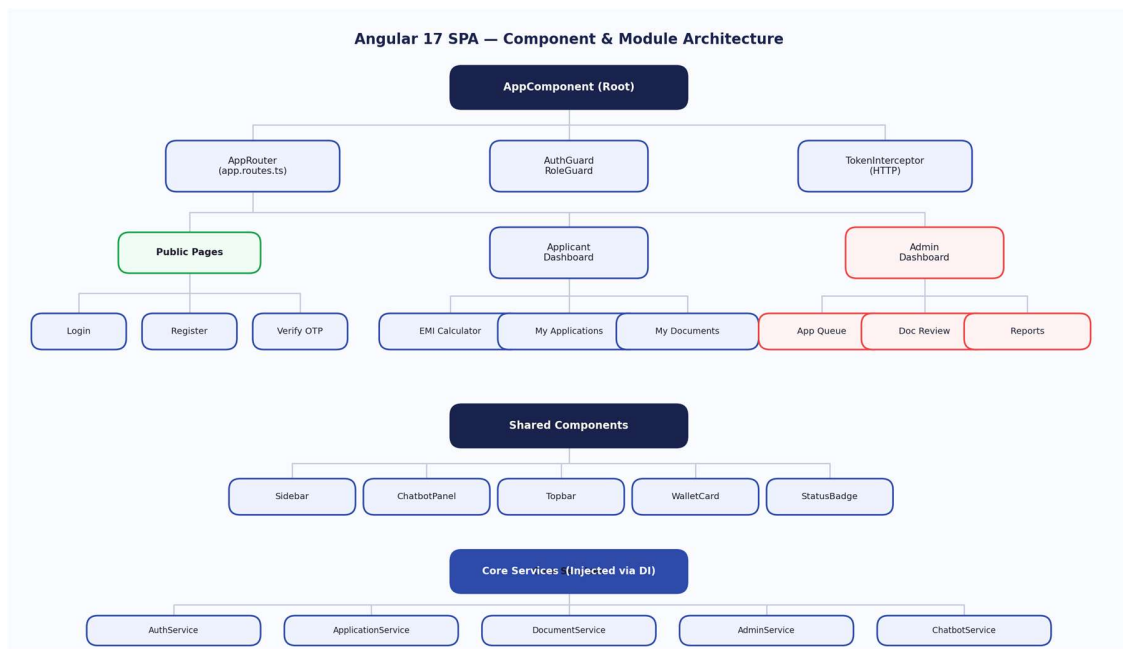
7.2 API Endpoints

Verb	Path	Auth	Description
GET	/api/admin/applications	Admin	Review queue with optional status filter
GET	/api/admin/applications/dashboard	Admin	Dashboard metrics
GET	/api/admin/applications/{id}	Admin	Application details
PUT	/api/admin/applications/{id}/status	Admin	Update status with review payload
GET	/api/admin/documents/application/{applicationId}	Admin	Proxy fetch docs for application
GET	/api/admin/documents?status=	Admin	Proxy fetch all docs
GET	/api/admin/documents/{id}/download	Admin	Proxy download document
PUT	/api/admin/documents/{id}/verify	Admin	Proxy verify document
GET	/api/admin/users	Admin	Proxy fetch users from auth service
PUT	/api/admin/users/{id}/status	Admin	Proxy update user status

7.3 Service Layer

- AdminLoanApplicationService for queue, dashboard, decision updates.
- Internal HTTP clients for AuthServiceClient and DocumentServiceClient.
- Event publisher and consumers for cross-service synchronization.
- MassTransit consumers:
 - ApplicationSubmittedConsumer
 - DocumentVerifiedConsumer
 - StatusSyncFailedConsumer
 - NotificationFailedConsumer

8. Frontend (Angular 21 SPA)



Angular Component Architecture

8.1 Application Bootstrap (app.config.ts)

- Router provider with standalone routes.
- HttpClient with fetch backend and interceptor chain.
- Interceptors: authInterceptor and errorInterceptor.
- Browser hydration and animations enabled.

8.2 Route Table (app.routes.ts)

Route	Area	Guard	Description
/	Public	None	Landing page
/login	Public	None	Login page
/signup	Public	None	Signup page
/applicant/dashboard	Applicant	authGuard + roleGuard(APPLICANT)	Applicant dashboard
/applicant/apply and /applicant/apply/{id}	Applicant	authGuard + roleGuard(APPLICANT)	Loan apply/edit flow
/applicant/applications	Applicant	authGuard + roleGuard(APPLICANT)	My applications list
/applicant/applications/{id}	Applicant	authGuard + roleGuard(APPLICANT)	Application detail
/applicant/applications/{id}/status	Applicant	authGuard + roleGuard(APPLICANT)	Status tracking
/applicant/applications/{id}/documents	Applicant	authGuard + roleGuard(APPLICANT)	App-specific documents
/applicant/documents	Applicant	authGuard +	My documents

		roleGuard(APPLICANT)	
/applicant/profile	Applicant	authGuard + roleGuard(APPLICANT)	Profile
/applicant/wallet	Applicant	authGuard + roleGuard(APPLICANT)	Applicant wallet
/admin/queue	Admin	authGuard + roleGuard(ADMIN)	Admin queue
/admin/documents	Admin	authGuard + roleGuard(ADMIN)	Admin documents
/admin/applications/{id}	Admin	authGuard + roleGuard(ADMIN)	Admin review screen
/admin/reports	Admin	authGuard + roleGuard(ADMIN)	Reports
/admin/users	Admin	authGuard + roleGuard(ADMIN)	User management
/admin/wallet	Admin	authGuard + roleGuard(ADMIN)	Admin wallet

8.3 Route Guards

- authGuard: checks authentication/token state.
- roleGuard(role): enforces required role for route tree.

8.4 HTTP Interceptors

- authInterceptor: appends bearer token to API requests.
- errorInterceptor: centralized frontend handling for API error payloads.

8.5 Core Services

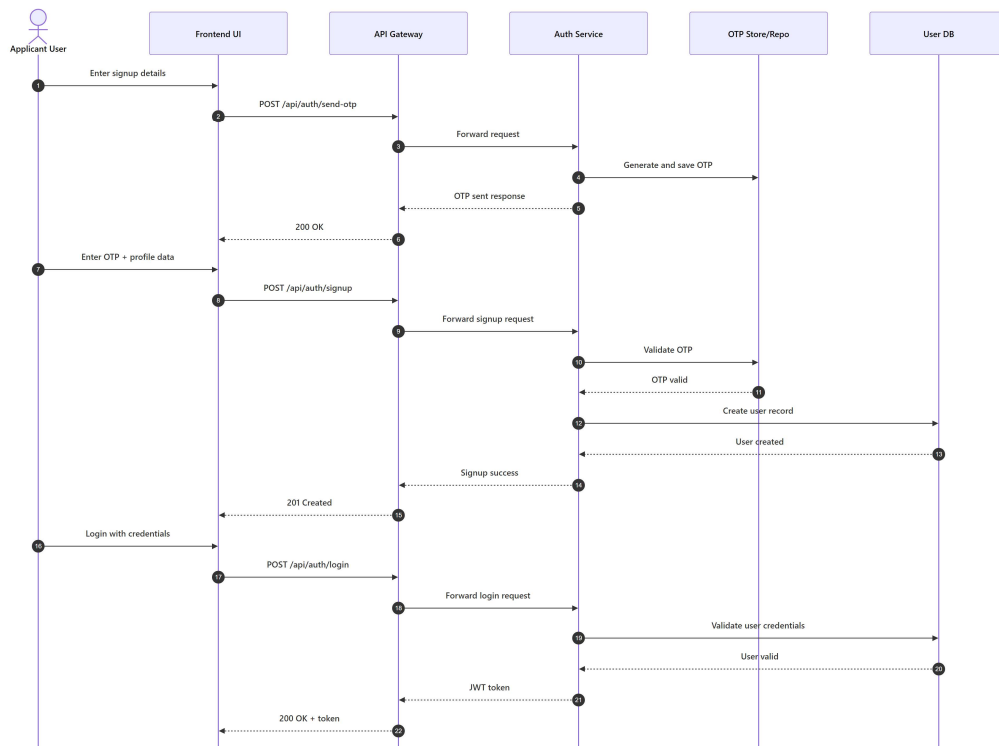
- AuthService: login/signup/token/user state.
- ApplicationService: profile, applications, wallet, status timeline.
- DocumentService: upload/list/replace/link/download operations.
- AdminService: queue, decision updates, admin-specific orchestration.
- Chat integration: gateway route based chatbot API usage.

8.6 Key Pages and Components

- Applicant pages: dashboard, apply-loan, my-applications, track-status, documents, wallet, profile.
- Admin pages: queue, documents, application-review, users, reports, wallet.
- Shared shell: layouts, sidebar, badges, cards, and reusable tables/forms.

9. Key Data Flows

9.1 Signup and OTP Verification



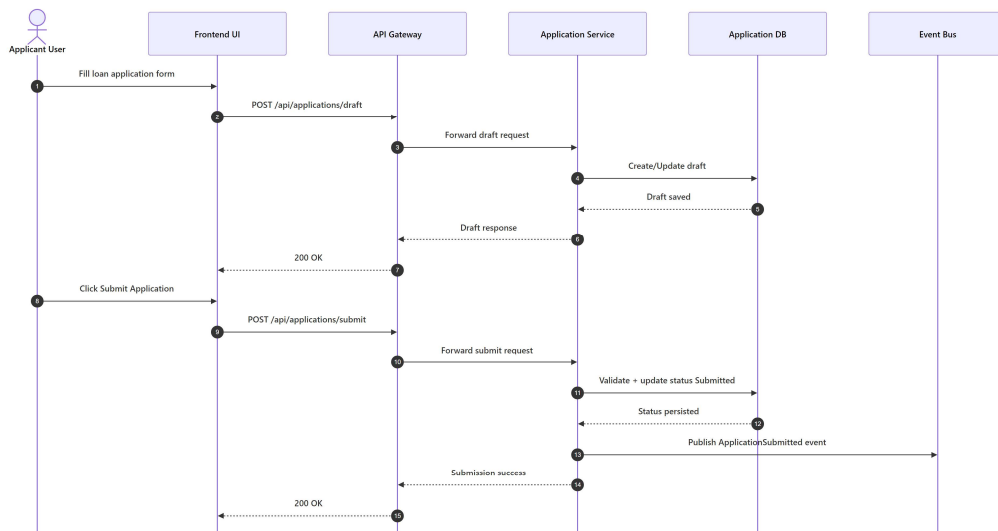
OTP Sign-up and Login

- 1. User starts signup.
- 2. Auth service sends OTP and stores verification state.
- 3. User verifies OTP.
- 4. Account is activated and downstream user-registration event is emitted.

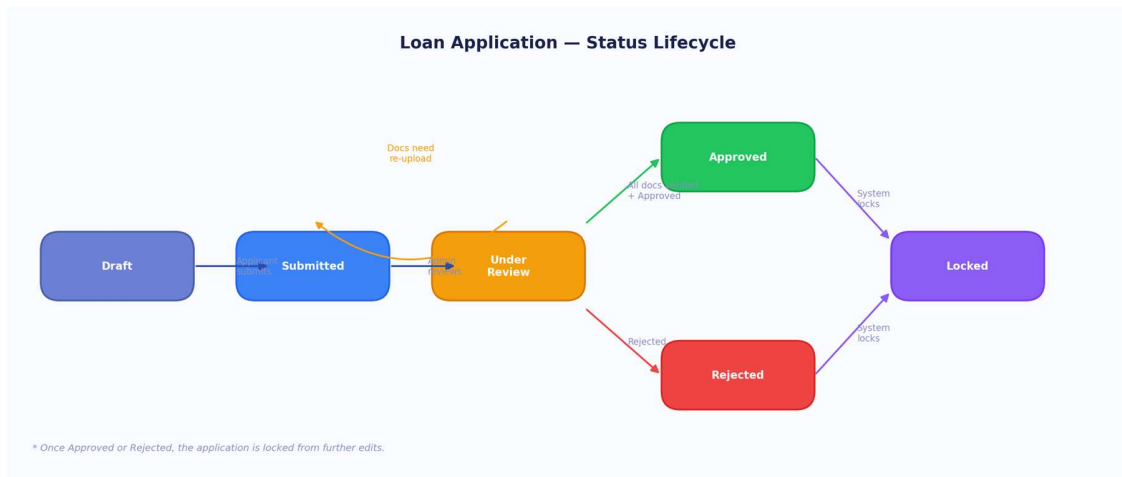
9.2 Login and JWT Propagation

- 1. User logs in.
- 2. Auth service issues JWT.
- 3. Frontend stores token and sends through auth interceptor.
- 4. Downstream services validate token and role claims.

9.3 Applicant Draft and Submission Flow



Application Submit Flow



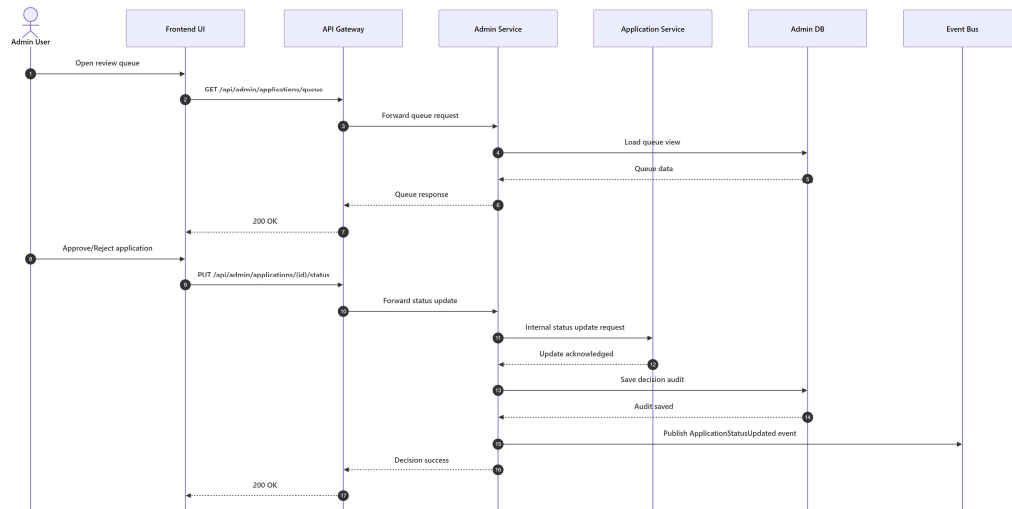
Application Status Lifecycle

- 1. Applicant saves profile and draft application.
- 2. Applicant uploads required documents.
- 3. Submit endpoint validates and transitions draft to submitted.
- 4. ApplicationSubmittedEvent is emitted for admin and notification flows.

9.4 Wallet Fee and Balance Validation Flow

- 1. Applicant submits loan application.
- 2. Application fee debit is attempted from applicant wallet.
- 3. If insufficient balance, service raises INSUFFICIENT_WALLET_BALANCE (HTTP 402).
- 4. Frontend keeps application as draft and asks user to top up wallet.

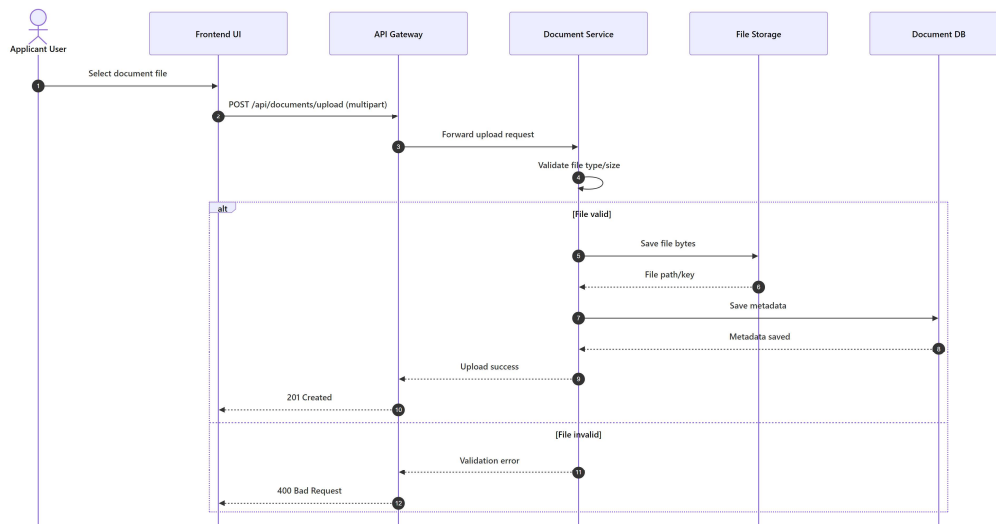
9.5 Admin Review and Disbursal Guard



Admin Review and Decision Flow

- 1. Admin opens queue and application details.
- 2. Admin updates status and decision fields.
- 3. Admin wallet sufficiency is validated before disbursal operations.
- 4. Status updates are synchronized and status-change events are emitted.

9.6 Document Verification Flow



Document Upload Flow

- 1. Applicant uploads/replaces document.
- 2. Admin verifies or requests reupload.
- 3. DocumentVerifiedEvent is emitted.
- 4. Application/admin/notification consumers react to status changes.

9.7 Chatbot Flow

- 1. Frontend calls gateway chat route.
- 2. Chatbot validates JWT and loads or creates session context.
- 3. Applicant flow: deterministic guided orchestration.
- 4. Admin flow: LLM summary response over preloaded admin context.

10. Infrastructure and Configuration

10.1 Docker Compose Services

Service	Container Port	Host Port	Notes
sqlserver	1433	1433	Persistent volume sqlserver_data
rabbitmq	5672 / 15672	5673 / 15673	Management UI exposed
auth-service	8080	5021	JWT + identity APIs
application-service	8080	5022	Loan + wallet APIs
document-service	8080	5023	Upload/verification APIs
admin-service	8080	5024	Admin APIs
notification-service	8080	5025	Event consumer + email
api-gateway	8080	5020	Ocelot entry point
chatbot-service	8000	5026	FastAPI chatbot
frontend	4200	4200	Angular UI

10.2 Database Configuration per Service

Service	Connection String Database
Auth Service	CapFinLoan_AuthDb
Application Service	CapFinLoan_ApplicationDb
Document Service	CapFinLoan_DocumentDb
Admin Service	CapFinLoan_AdminDb

10.3 JWT Configuration

Shared keys used across services:

- Jwt:Key
- Jwt:Issuer
- Jwt:Audience
- Jwt:ExpiryMinutes

10.4 RabbitMQ and MassTransit

- RabbitMQ host, username, password configured per service.
- Keabab-case endpoint naming formatters scoped by service prefixes.

- ConfigureEndpoints automatically wires consumers to queues.

10.5 Runtime Health and Startup

- Docker compose health checks for SQL, RabbitMQ, and API services.
- Service startup depends on health of dependencies.
- APIs run schema migrations on startup in service Program.cs.

11. Unit Test Projects

Test Project
CapFinLoan.Auth.UnitTests
CapFinLoan.Application.UnitTests
CapFinLoan.Document.UnitTests
CapFinLoan.Admin.UnitTests
CapFinLoan.Notification.UnitTests
ChatbotService tests/test_orchestrator.py

11.1 Backend Testing Framework

- .NET test projects validate service logic, exception behavior, and domain transitions.
- Messaging and orchestration paths are validated at service layer boundaries.

11.2 Frontend and Chatbot Testing

- Frontend test config present at tsconfig.spec.json.
- Chatbot has orchestrator unit tests covering guided conversation and payload construction.

12. Security Considerations

- Password handling through ASP.NET Identity in Auth service.
- JWT validated independently in each API service.
- Role-based authorization annotations on controllers and routes.
- Admin-only internal endpoints protected via role constraints.
- Internal notification lookup endpoint additionally guarded by X-Internal-API-Key.
- Document upload/download paths enforce user ownership or admin role checks.
- Wallet operations enforce server-side balance checks and exception-safe debits.